

USB 読本

LDLabo

2005 年 3 月 2 日

目次

第 1 章	USB の概要	1
1.1	信号	1
1.2	通信	2
1.3	接続構造	2
1.4	基本単位	2
1.5	パケットの種類	3
1.6	フレーム	4
1.7	Token パケット	4
1.8	Token パケットへの応答	4
1.9	パケットのエラー処理	5
1.10	デバイスの設定	5
1.11	デバイスの接続	6
1.12	基本手順	6
1.13	転送方式の選択	7
1.14	コントロール転送	7
1.15	ディスクリプタ	7
1.16	標準デバイスリクエスト	8
1.17	ディスクリプタの構成	9
1.18	転送の種類	13
1.19	情報源	15

図目次

1.1	NRZI	2
1.2	構造	2
1.3	パケット	3
1.4	PID	3
1.5	フレーム	4
1.6	パケット	5
1.7	基本手順	6
1.8	コントロール転送手順	8
1.9	ディスクリプタ構成 例 2	12
1.10	ディスクリプタ構成 例 1	14

表目次

1.1	信号線	1
1.2	パケットの種類	3
1.3	標準デバイスリクエスト	10
1.4	※ 1	10
1.5	※ 2	10
1.6	※ 3	11
1.7	bmRequestType	11
1.8	デバイスディスクリプタ	11
1.9	コンフィギュレーションディスクリプタ	12
1.10	インターフェースディスリプタ	12
1.11	エンドポイントディスリプタ	13
1.12	ストリングディスリプタ	13
1.13	ディスリプタの形式	13
1.14	転送の種類	15

第 1 章

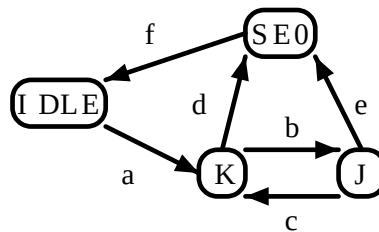
USB の概要

1.1 信号

USB は 2 本の電源供給線と 2 本の信号線を持っています。信号線は双方向の電流駆動で平衡型回路になっています。電流は 10.71mA 以上の駆動を必要とします、電流を駆動した場合に論理値 1 または 0 を示します。電流駆動がなされない場合は論理値は無しになります。この場合はプルアップまたはプルダウンされた抵抗で電位が定まりますが、このときは IDLE 状態であつ、full-speed と low-speed を選ぶ転送速度の決定などに使われています。

表 1.1 信号線

遷移	full		low		意味
	D+	D-	D+	D-	
a	0	1	1	0	SOP
b	1	0	0	1	J
c	0	1	1	0	K
d	0	0	0	0	EOP
e	0	0	0	0	EOP
f	1	0	0	1	IDLE



まとめると

表 1.1 の様になります。電位だけで考えると意味が重なるものがありますが、これらは状態位置が違うので区別できます。信号線はホストとデバイスの間をつないでいます、デバイスのつながっ

ていない信号線はホスト側の $15\text{K}\Omega$ でプルダウンされているので $D+=0, D-=0$ です。full-speed のデバイスが繋がると $1.5\text{K}\Omega$ のプルアップで $D+=1, D-=0$ になります、この電位が 2ms 以上続くとホストが接続を認識します。low-speed のデバイスが繋がると $1.5\text{K}\Omega$ のプルアップ $D+=0, D-=1$ になります、この電位が 2ms 以上続くとホストが接続を認識します。 $D+=0, D-=0$ が 10ms 以上続いた場合は Reset です。

- IDLE : ホストとデバイスがつながっているがパケットは転送されていないとき。
- J : パケット中の論理値 1
- K : パケット中の論理値 0
- SE0 : Single ended 0 ホストが 10ms 以上駆動した場合は Reset、デバイスが外されたとき 2.5us 続いたらホストがデバイスの取り外しを認識します。
- SOP : Start of Packet - パケットの開始で IDLE から K になったときです。
- EOP : End of Packet - パケットの終了で送り出し側では 2 ビット期間送ります、受け取り側では 1 ビット

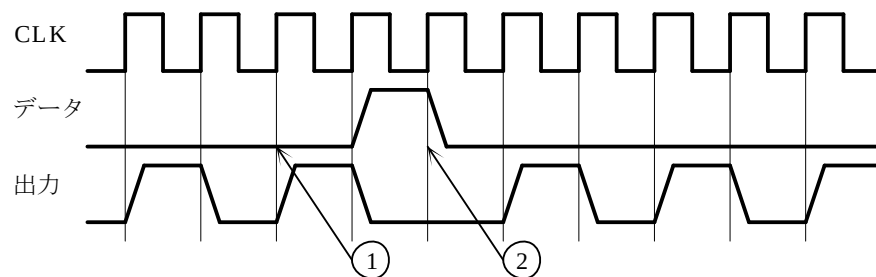
期間受け取ったらパケットの終了を認識します。

本稿は自己解釈によるものです、出典は USB 1.1 の仕様書 *Universal Serial Bus Specification Revision 1.1* の p.107 の *Chapter 7 Electrical* にあります。

1.2 通信

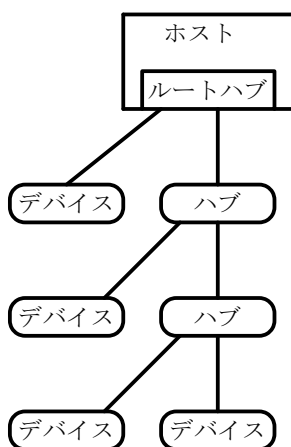
半 2 重のシリアル通信です、一度に送信と受信はできません。通信データは NRZI に変換されています、この方式では 1 が続くと信号の変化点が無くなりますが、USB では送信側で NRZI 変換前のビット列が 1 で 6 個続いた場合は元のデータとは無関係に 0 を挿入します。受信側では NRZI から逆変換後のビット列が 6 個続いた後の 0 はデータと無関係のビットとして捨てます。データが 0 から 1、1 から 0 への変化点は受信側がデータを取得するためのクロックを送信側のクロックに同期するために使います。USB では一定期間内に必ずビットデータに変化点があるようにすることで、受信側がデータから取得クロックを作ることができるようにしています。

図 1.1 NRZI



NRZI は図 1.1 の様にデータが 1 のときは出力をそのまま②、0 のときは出力を反転①します。受信側が 1 ビットの 10 倍の周波数で出力をサンプリングしているのならば、出力の変化点で計数器を初期化すると 5CLK 目の位置が出力ビットのほぼまんなかに位相が合うようになります。

図 1.2 構造



1.3 接続構造

通信線は図 1.2 の様に唯一のホストに対して多数のハブ（ハブデバイス）とデバイスが接続されたバス型になっていることを前提に考えます。デバイスはハブに接続されます、ハブはハブどうしでルートハブは含まずに 5 段の深さまで接続できます。最初のハブはホストと一体になったルートハブと呼ばれるものです。

1.4 基本単位

通信の基本単位はパケット (Packet) です。パケットはビット同期を取るための SYNC から始まって両信号端子を 0V にする EOP(End of Packet) で終わります。EOP はデータの一部ではなく信号端子の状態なので、同期が取れていなくても認識できるようになっています。パケットはバイトデータの集まったものです、バイトデータは LSB から通信線に送り出されます、SYNC は 80h なのでビットデータは 0000 0001 と送りだされます。受信側では 1 を待ってビットの取得を始めると SYNC の次のバイト

データのビットから取り込めるようになります。SYNC の 0000 0001 は NRZI 変換すると 1010 1011 になるので受信側はこの期間にビット同期を確立させておきます。仮に同期確立が出来なくても、パケットには CRC が付けられているので、データが読めなかったことは分かります、さらに同期がとれていなくてもパケットの最後の認識は先の理由のできるので通信の基本はしっかりしたものになっています。

図 1.3 パケット



パケットの基本的な構造は図 1.3 の様に頭に SYNC がついて同期を取ります、次の PID がパケットの種類を示します、PID は自身の検査データも含んでいます、次に内容が続き内容から計算した CRC を付けて終わります。Data パケット以外ではパケットの大きさは固定ですが Data パケッ

トは内容の大きさによってパケットの大きさが変わります、しかし、データの数パケット自体には含まれていません。パケットの最後には論理値を取らない特別な状態の EOP になりますから、その直前は CRC と分かります、CRC を引いた残りが内容と言うことになります。

1.5 パケットの種類

パケットの種類は SYNC に続く 1 バイトの PID で示します、ただし図 1.4 の様に上位の 4 ビットは下位の 4 ビットを反転したものですので 4 ビットで 16 種類を区別できます。上位の 4 ビットが下位の 4 ビットの反転したものと合致しなかった場合はパケットに対応する処理ができないことが早々に認識できます。パケットは

表 1.2 の様に 10 種類が定義されています、これらは 4 形式に分けられます。Token と Special はホストだけが送信できるパケットです。Data と Handshake はホストとデバイスの双方が送受信するパケットです。

表 1.2 パケットの種類

形式	名前	PID	バイト数	CRC	説明
Token	OUT	0001	4	5	Data パケットの送信要求
	IN	1001	"	"	Data パケットの受信要求
	SOF	0101	"	"	フレームの開始
	SETUP	1101	"	"	デバイスリクエスト
Data	Data0	0011	可変	16	偶数番目パケット
	Data1	1011	"	"	奇数番目パケット
Handshake	ACK	0010	2	なし	肯定応答
	NAK	1010	"	"	否定応答
	STALL	1110	"	"	停止中
Special	PRE	1100			

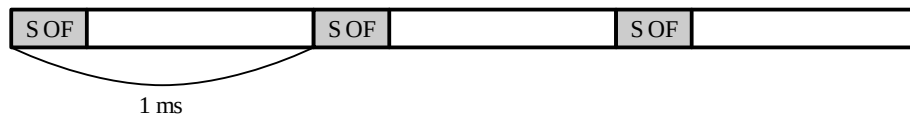
図 1.4 PID



パケットの内容の内訳はパケットの形式によって違ってきます。Token 形式のパケットの場合は図 1.6 の様に内容は 11 ビットなので 5 ビットの CRC *1 を合わせて 2 バイトになるようになっています。Data パケットの内

*1 $G(x) = x^5 + x^2 + 1$

図 1.5 フレーム



容のバイト数は low-speed で 0～8 バイト、full-speed で 0～1023 バイトです。Data パケットには 16 ビットの CRC ^{*2} が使われています。内容のバイト数が 0 という Data パケット ^{*3} も実際に使われます。Handshake パケットは Data パケットへの応答に使われるもので内容と CRC はありません、PID がその機能を達します。

1.6 フレーム

通信はホストが主導して行うのは先のとおりです。ホストは通信線を時分割するために図 1.5 の様に 1ms 毎に通信線に SOF(Start of frame) パケットを送り出しています。このパケットに対してデバイスが通信線に応答することはありません、この期間をフレームと呼んでいます、SOF には 11 ビットのフレーム番号が付けられています。SOF は Token パケットに属しています。ホストはフレームの中でやりとりするパケットの数を制御することで特定のデバイスの単位時間のデータ転送数を保障することができます。

1.7 Token パケット

Token に属するパケットは他にホストがデバイスに対してデータを送ることを要求する IN パケットとホストがデバイスに対してデータを受けることを要求する OUT パケットがあります。IN と OUT のパケットはアドレスを 7 ビット持っているのでデバイスは常に Token パケットを監視して自分のアドレスが指定されているかどうかを把握する必要があります。USB に接続されたデバイスはホストからアドレスをひとつ貰っています、デバイスが複数のチャンネルを必要とする場合には IN と OUT パケットが持つ 4 ビットの ENDP (エンドポイント) を使うことができます。SETUP パケットはデバイスリクエストを発行するためのものでコントロール転送の開始を知らせるものでもあります。

1.8 Token パケットへの応答

Token パケットは、その働きからホストからデバイスに対する問いかけ ”モシモシ” に当たるようです。デバイスはパケットのアドレスから自分に対する問いかけであるかどうかを知ります。パケットが OUT パケットならデバイスはパケットで指定されたエンドポイントにデータを受ける必要があります。このデータは 16 ビット時間以内に次の Data パケットで送られてきます。デバイスがデータ受け取ったら応答を返す場合があります、これは Handshake パケットを使います、応答が必要かどうかは転送の種類で決まっています。Handshake パケットは 3 種類あります、肯定応答の ACK、否定応答の NAK、停止中の STALL があります。ACK で応答した場合には次のデータがあれば、次の Data パケットで送られてきます、無ければデータ転送の終了です。NAK で応答した場合には Data パケットが再送されてきます。Token パケットが IN パケットだった場合には Data パケットはデバイスが送り応答はホストが返してきます。

^{*2} $G(x) = x^{16} + x^{15} + x^2 + 1$

^{*3} コントロール転送のステータスステージの Data パケット

図 1.6 パケット

Token

SYNC								PID								Fr a me												CRC5					EOP
0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7		
0	0	0	0	0	0	0	1	P0	P1	P2	P3	$\overline{P0}$	$\overline{P1}$	$\overline{P2}$	$\overline{P3}$																		

Token (SETUP, IN, OUT)

SYNC								PID								ADDR								ENDP					CRC5					EOP
0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7			
0	0	0	0	0	0	0	1	P0	P1	P2	P3	$\overline{P0}$	$\overline{P1}$	$\overline{P2}$	$\overline{P3}$																			

Data

SYNC								PI D								DATA								EOP
0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7	
0	0	0	0	0	0	0	1	P0	P1	P2	P3	$\overline{P0}$	$\overline{P1}$	$\overline{P2}$	$\overline{P3}$									

	DATA								CRC16																EOP
	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7	

Handshake

SYNC								PID								EOP
0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7	
0	0	0	0	0	0	0	1	P0	P1	P2	P3	$\overline{P0}$	$\overline{P1}$	$\overline{P2}$	$\overline{P3}$	

1.9 パケットのエラー処理

パケットのエラーはパケットが壊れているので、そのパケットを無視する場合と、パケットを待っているのに到着しない場合の2種類があります。壊れているとはPIDのチェックが合わない場合とCRCが合わない場合です。到着しないとは、送られる必要のあるパケットが16ビット時間以内に來ないときです。これは18ビット時間以内にタイムアウト処理になります。

1.10 デバイスの設定

Token パケットに示されるデバイスのアドレスは予めホストから与えられたものですから、これが与えられるまでは通常の通信はできません。接続されたばかりのデバイスはホストからアドレスが与えられるまでは0のアドレスに応答することになっています。USB では転送方法の種類がいくつかあります、デバイスはすべての種類に対応する必要はありませんが唯一コントロール転送には対応している必要があります、なぜならアドレスの割り当てはコントロール転送で送られるからです。デバイスがどの転送に対応できるかなどの情報もホストがデバイスから読み取りますが、これらもコントロール転送で行います。デバイスは単にデータ転送を行うだけでなくホストがデバイスとの通信に必要な情報を持っていてホストの要求があれば送り出す必要もあります。構成前のデバイスはアドレス0、エンドポイント0でホストとコントロール転送を行いデバイスリクエストにより構成を完了します。

1.11 デバイスの接続

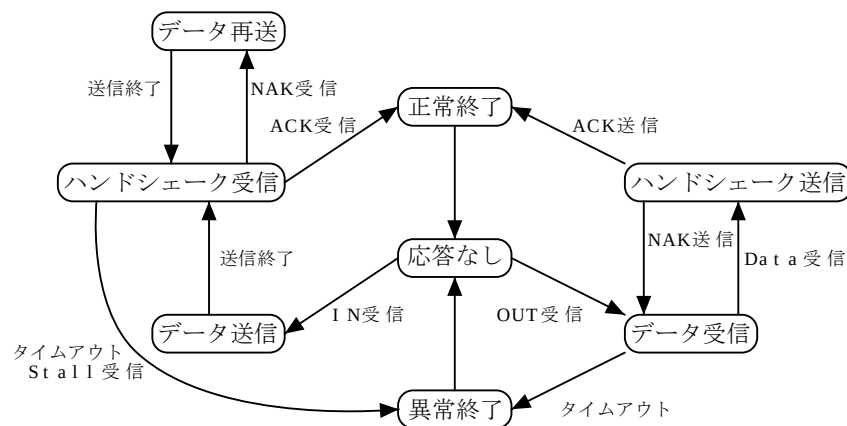
デバイスが最初に接続されたときはアドレスは 0 でコントロール転送で通信ができるようになっていますが、USB1.1 では通信速度がロースピードとフルスピードの 2 種類がありますので、ホストがデバイスの対応している速度で通信しないとホストがデバイスとの通信条件を整えることができません。このため通信速度はデバイスが 2 本ある通信線のどちらかをプルアップすることで、ホストは通信線がデータ転送していないときはいつでもデバイスの対応できる速度を知ることができるようになっています。まとめると

1. デバイスが接続されていないときは通信線は両方ともホスト側でプルダウンされているので 0V になっています。
2. デバイスが接続されると通信線のどちらかがプルアップされるので、新しいデバイスの接続がホストに認識されます。
3. ホストは新しいデバイスの接続と、その通信速度を認識します。
4. ホストは接続の認識から 100ms の間においてデバイスの安定を待ってから通信線を 10ms の間 0V にしてデバイスを初期化します。
5. ホストとデバイスは通信条件の整備を行うための情報のやり取りをコントロール転送で行いデバイスはホストからアドレスを付与されます。

1.12 基本手順

デバイス側から見た基本的な通信の手順を図 1.7 に書いてみました。デバイスは自分あての Token パケットが送られてくるまでは待機していて「応答なし」の状態です。SETUP パケットを受けるとコントロール転送に移行しますが、これは後述します。コントロール転送以外では Token パケットを受けると Data パケットが転送されて Data パケットを受けたホストあるいはデバイスが Handshake パケットの ACK または NAK または STALL のどれかで応答します。

図 1.7 基本手順



NAK パケットで応答したときは Data パケットを再送しますが、エラーが続いても何回まで再送するかは当方は未確認です。デバイスが Data パケットを受けているとき NAK に応答がない場合はタイムアウトで「異常終了」から「応答なし」に戻します。デバイスがホストに Data パケットを送っているときに何回も NAK で再送

が要求される場合に再送数に上限を設けて Data パケットを送らなければホストがタイムアウト処理に入ります。

1.13 転送方式の選択

いくつかの転送方式がありますが、どの転送方式になるかは、先のデバイス接続時のやりとりでデバイスから読み取られる情報に入っています、どのデバイスのどのエンドポイントは、どの転送方式と個別に認識されています。ただしエンドポイント 0 はコントロール転送用で他の転送方式には割り当てられません。エンドポイント 0 の情報はパケットの最大容量だけです。コントロール転送はホストとデバイス間の通信条件を整えるときだけに使うようで、通常のデータ転送には使わないようです。

1.14 コントロール転送

転送方式は 4 種類ありますが、コントロール転送は先の理由から通常のデータ転送には使われませんし、手順も他の方式よりも独特のものになっています。通信は 1ms の時分割されたフレーム単位で行いますが、コントロール転送の説明では他の転送方式では用いない STAGE(ステージ) という言葉を使っています。すなわち、コントロール転送は 3 ステージで構成されます。

1. Setup ステージ

それぞれのステージは別々のフレームに置かれています。Setup ステージは

2. Data ステージ

Token パケットの SETUP を使います、このパケットの次にはホストが Data

3. Status ステージ

ケットの Data0 をさらに続けます。この Data パケットの内容は デバイスリク

エストと言うもので 8 バイトあります。デバイスはこの内容を読み取って自身

の状態を変更したり、情報を返したりします。ただし、状態を変更するのは Status ステージの終了後であり、情報を返すのは Data ステージで行います。Data ステージは Setup ステージの次のエンドポイント 0 を指す Token の IN か OUT のパケットに続ける Data パケットです。Status ステージはデータ数が 0 の Data パケットを受け取ったときに Status ステージと認識します。デバイス側から見たコントロール転送の手順を図 1.8 に書いてみました。3 ステージで転送するので図 1.7 より複雑に見えますが同じです。

1.15 ディスクリプタ

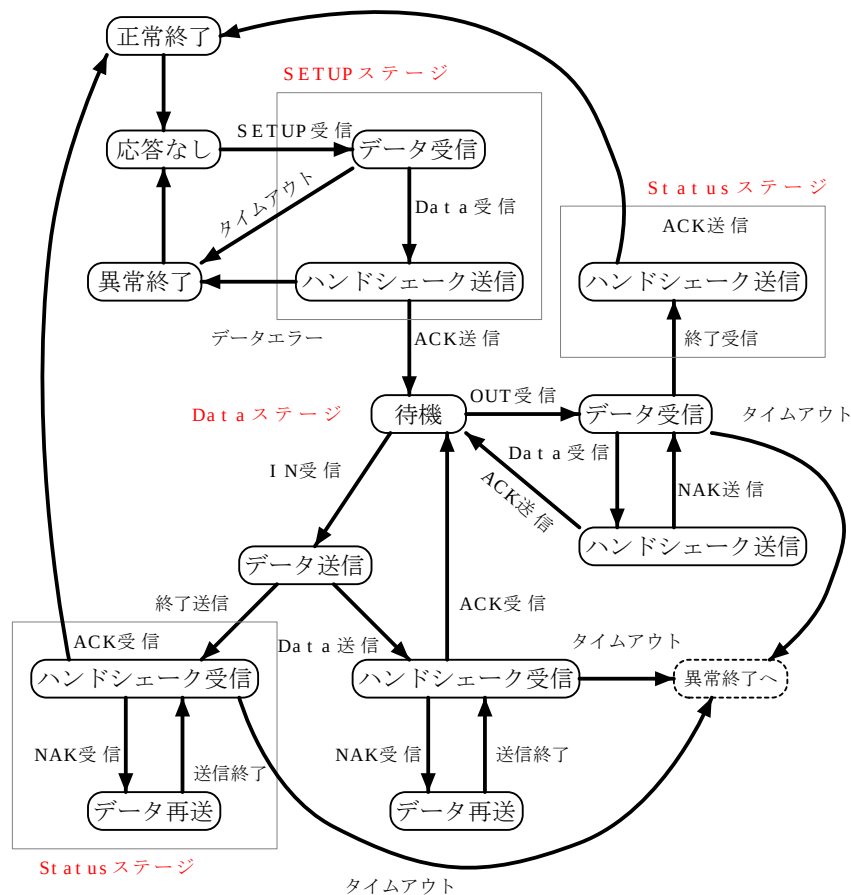
コントロール転送ではホストはデバイスの持つディスクリプタを読みます。デバイスはディスクリプタを記憶するメモリまたはレジスタが必要です。デバイスディスクリプタはデバイスに関する一般情報を書いてあるもので、デバイスごとにひとつだけあるものです。エンドポイントディスクリプタは(エンドポイントの)アドレス、転送方式、容量などを書いてあります。USB を説明した資料にはコンフィギュレーションを構成と言っているものがあるのでここはその言葉も使いながら説明を続けます。

- デバイスディスクリプタ (表 1.8)
- コンフィギュレーションディスクリプタ (表 1.9)
- インターフェースディスクリプタ (表 1.10)
- エンドポイントディスクリプタ (表 1.11)
- スtringディスクリプタ (表 1.12)

ホストからデバイスを見た場合には、それぞれはエンドポイントのかたまりのように見えています。エンドポイント個々の特性は先のエンドポイントディスクリプタに書いてあります、例外としてエンドポイント 0 に対応するエンドポイントディスクリプタはありません、エンドポイントが 1 種類以上の転送方式をもっている場合もあります、エンドポイントを場面に応じた特性で使い分けることもあります。

す。エンドポイントの使い方を決定することをコンフィギュレーションといいます、コンフィギュレーションす

図 1.8 コントロール転送手順



る前を構成が選択されていないといいますが、コンフィギュレーション後を構成されているといいますが。コンフィギュレーションは専用のコマンドが用意されていて、そこでコンフィギュレーションディスクリプタを指定してデバイスが可能な構成の中からホストが任意で選んだものでデバイスのエンドポイントを構成します。そういうことなのでデバイスはコンフィギュレーションディスクリプタを最低でもひとつは必要ですし、いくつもの構成を提供するのなら、同数のコンフィギュレーションディスクリプタを持つ必要があります。コンフィギュレーションディスクリプタの下位の構造としてインターフェースディスクリプタがあります、インターフェースディスクリプタはエンドポイントディスクリプタを下位に持っていて、ここに書かれた内容でエンドポイントの使い方が決まります。インターフェースディスクリプタとエンドポイントディスクリプタはコンフィギュレーションディスクリプタを構成するものなのでコンフィギュレーションディスクリプタの全体と言った場合には下位のこれらのディスクリプタを含みます。STRINGディスクリプタは文字列を格納しているところです、各ディスクリプタから番号で参照されています。

1.16 標準デバイスリクエスト

基本的に表 1.3 の様なデバイスの持つディスクリプタの情報の読み取り、書き換えや、その情報を基に行われる操作などがあります。

- Get Descriptor

指定したディスクリプタの読み取りを行います。指定できるディスクリプタはデバイスディスクリプタ、コンフィギュレーションディスクリプタ、ストリングディスクリプタです。指定の方法はディスクリプタの形式と番号です、形式は先に挙げた3種類です。コンフィギュレーションディスクリプタを選択した場合はそれを構成するインターフェースディスクリプタ、エンドポイントディスクリプタも含んでいます。番号と言うのは、例えばコンフィギュレーションディスクリプタが5個あるとすれば番号は0から4でそれらを指定できるという具合です。

- Set Descriptor

ディスクリプタの更新または追加を行います、ディスクリプタの指定は先と同じく形式と番号で行います。

- Set Configuration

デバイス構成の設定を行います。構成の選択はコンフィギュレーションディスクリプタから得られている構成値 (bConfigurationValue) で指定します。構成値を0にしたときはデバイスを未構成に戻すことになります。

- Get Configuration

現在のデバイス構成を得ます。デバイス構成とは構成値のことです、0の場合は先にも触れたように未構成を示します。

- Set Address

デバイスにアドレスの割り当てを行います。

他にもありますが、ここでは重要と思われるものを取り上げました。

1.17 ディスクリプタの構成

デバイスはディスクリプタを持ちホストはデバイスからディスクリプタを読み出してデバイスとそのエンドポイントの使い方を把握します。ホストが読み出すディスクリプタはデバイスディスクリプタ、コンフィギュレーションディスクリプタ、ストリングディスクリプタの3種類です。

■ デバイスディスクリプタはデバイスにひとつだけあります、コンフィギュレーションディスクリプタとストリングディスクリプタは複数個あるので番号を指定して読み出します。コンフィギュレーションディスクリプタがいくつあるかはデバイスディスクリプタの bNumConfigurations に示されます。

■ コンフィギュレーションディスクリプタを読み出すとその構成に必要なすべてのインターフェースディスクリプタとエンドポイントディスクリプタが同時に読み出されます。コンフィギュレーションディスクリプタには、その構成で取り扱うインターフェースの数が bNumInterface に示されます。ホストが読み取ったコンフィギュレーションディスクリプタを含む構成全体からインターフェースディスクリプタを特定するのはインターフェースディスクリプタの bInterfaceNumber で示されるインターフェース番号と代替値です。

■ インターフェースディスクリプタには、そのインターフェースが取り扱うエンドポイントの数が bNumEndpoints に示されます。ホストが読み取ったコンフィギュレーションディスクリプタを含む構成全体からエンドポイントディスクリプタを特定するのはエンドポイントディスクリプタの bEndpointAddress で示されるエンドポイント番号です。

機能名	bmRequestType 1 バイト	bRequest 1 バイト	wValue 2 バイト	wIndex 2 バイト	wLength 2 バイト	Data
CLEAR_FEATURE	0000 0000 0000 0001 0000 0010	1	選択値	0 Interface Endpoint	0	なし
GET_CONFIGURATION	1000 0000	8	0	0	1	構成値
GET_DESCRIPTOR	1000 0000	6	Type:Index	0 か言語 ID	バイト数	ディスクリプタ
GET_INTERFACE	1000 0001	10	0	Interface	1	代替値
GET_STATUS	1000 0000 1000 0001 1000 0010	0	0	0 Interface Endpoint	2	※ 1 表 1.4 ※ 2 表 1.5 ※ 3 表 1.6
SET_ADDRESS	0000 0000	5	アドレス	0	0	なし
SET_CONFIGURATION	0000 0000	9	構成値	0	0	なし
SET_DESCRIPTOR	0000 0000	7	Type:Index	0 か言語 ID	バイト数	ディスクリプタ
SET_FEATURE	0000 0000 0000 0001 0000 0010	3	選択値	0 Interface Endpoint	0	なし
SET_INTERFACE	0000 0001	11	代替値	Interface	0	なし
SYNCH_FRAME	1000 0010	12	0	Endpoint	2	フレーム番号

Type (上位バイト): ディスクリプタ形式 表 1.13 参照
Index (下位バイト): ディスクリプタ番号
Interface: インターフェースディスクリプタ番号
Endpoint: エンドポイントディスクリプタ番号

D7	D6	D5	D4	D3	D2	D1	D0
0						リモート・ウェイクアップ	自己電源
D7	D6	D5	D4	D3	D2	D1	D0
0							

D7	D6	D5	D4	D3	D2	D1	D0
0							
D7	D6	D5	D4	D3	D2	D1	D0
0							

表 1.6 ※ 3

D7	D6	D5	D4	D3	D2	D1	D0
0							ストール
D7	D6	D5	D4	D3	D2	D1	D0
0							

表 1.7 bmRequestType

定義	ビット	値	意味
転送方向	D7	0	ホスト→デバイス
		1	デバイス→ホスト
形式	D6,D5	0	Standard
		1	Class
		2	Vendor
		3	Reserved
内容	D4~D0	0	Device
		1	Interface
		2	Endpoint
		3	Other
		4~31	Reserved

表 1.8 デバイスディスクリプタ

定義名	バイト数	意味
bLength	1	ディスクリプタのバイト数
bDescriptorType	1	ディスクリプタの形式
bcdUSB	2	USB 仕様リリース番号 (BCD)
bDeviceClass	1	クラスコード
bDeviceSubClass	1	サブクラスコード
bDeviceProtocol	1	プロトコルコード
bMaxPacketSize0	1	エンドポイント 0 の最大パケットサイズ
idVendor	2	ベンダ ID
idProduct	2	プロダクト ID
bcdDevice	2	デバイスリリース番号 (BCD)
iManufacturer	1	製造者を示すstringディスクリプタ番号
iProduct	1	製品を示すstringディスクリプタ番号
iSerialNumber	1	製造番号を示すstringディスクリプタ番号
bNumConfigurations	1	可能な構成の数

表 1.9 コンフィギュレーションディスクリプタ

定義名	バイト数	意味
bLength	1	ディスクリプタのバイト数
bDescriptorType	1	ディスクリプタの形式
wTotalLength	2	本構成の全ディスクリプタのバイト数
bNumInterface	1	本構成でサポートされるインターフェースの数
bConfigurationValue	1	構成値
iConfiguration	1	本構成を示すストリングディスクリプタ番号
bmAttributes	1	本構成の特性 D7 : 1 D6 : 自己電源 D5 : リモート・ウェイクアップ D4~D0 : 0
MaxPower	1	最大電力消費量 2mA 単位

表 1.10 インターフェースディスクリプタ

定義名	バイト数	意味
bLength	1	ディスクリプタのバイト数
bDescriptorType	1	ディスクリプタの形式
bInterfaceNumber	1	インターフェース番号
bAlternateSetting	1	代替値
bNumEndpoints	1	本インターフェースの構成するエンドポイントの数
bInterfaceClass	1	クラスコード
bInterfaceSubClass	1	サブクラスコード
bInterfaceProtocol	1	プロトコルコード
iInterface	1	本インターフェースを示すストリングディスクリプタ番号

■ エンドポイントディスクリプタは同一構成の中のインターフェースで重複定義することは許されないのでホストが読み取ったコンフィギュレーションディスクリプタを含む構成全体の中には該当するエンドポイント番号のエンドポイントディスクリプタはひとつだけです。尚、エンドポイントディスクリプタは 0 から 15 までの 16 個まで持つことが出来ます、エンドポイントの 0 番はコントロール転送でデバイスリクエストを受ける必須のものです、どの構成を選んでもエンドポイントの 0 番はコントロール転送に当てられるのでエンドポイントディスクリプタはありません。

■ ディスクリプタの構成例を 2 例示します、図 1.10 は構成を 2 個持っていて、どちらかを選択できます、コンフィギュレーション 1 のインターフェース 2 は代替値で 2 種類から選択できます。図 1.9 は最も簡単なもの

図 1.9 ディスクリプタ構成例 2

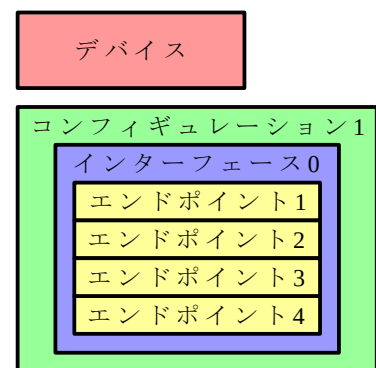


表 1.11 エンドポイントディスクリプタ

定義名	バイト数	意味
bLength	1	ディスクリプタのバイト数
bDescriptorType	1	ディスクリプタの形式
bEndpointAddress	1	エンドポイントのアドレス D7 : 方向 (OUT 0 / IN 1) D6~D4 : 0 D3~D0 : エンドポイント番号
bmAttributes	1	エンドポイントの属性 0 : コントロール転送 1 : アイソクロナス転送 2 : バルク転送 3 : インタラプト転送
wMaxPacketSize	2	本エンドポイントのパケットの最大データ数
bInterval	1	ポーリング間隔 アイソクロナス転送は 1、インタラプト転送は 1~255

表 1.12 スtringディスクリプタ

定義名	バイト数	意味
bLength	1	ディスクリプタのバイト数
bDescriptorType	1	ディスクリプタの形式
bString	—	UNICODE の文字列

表 1.13 ディスクリプタの形式

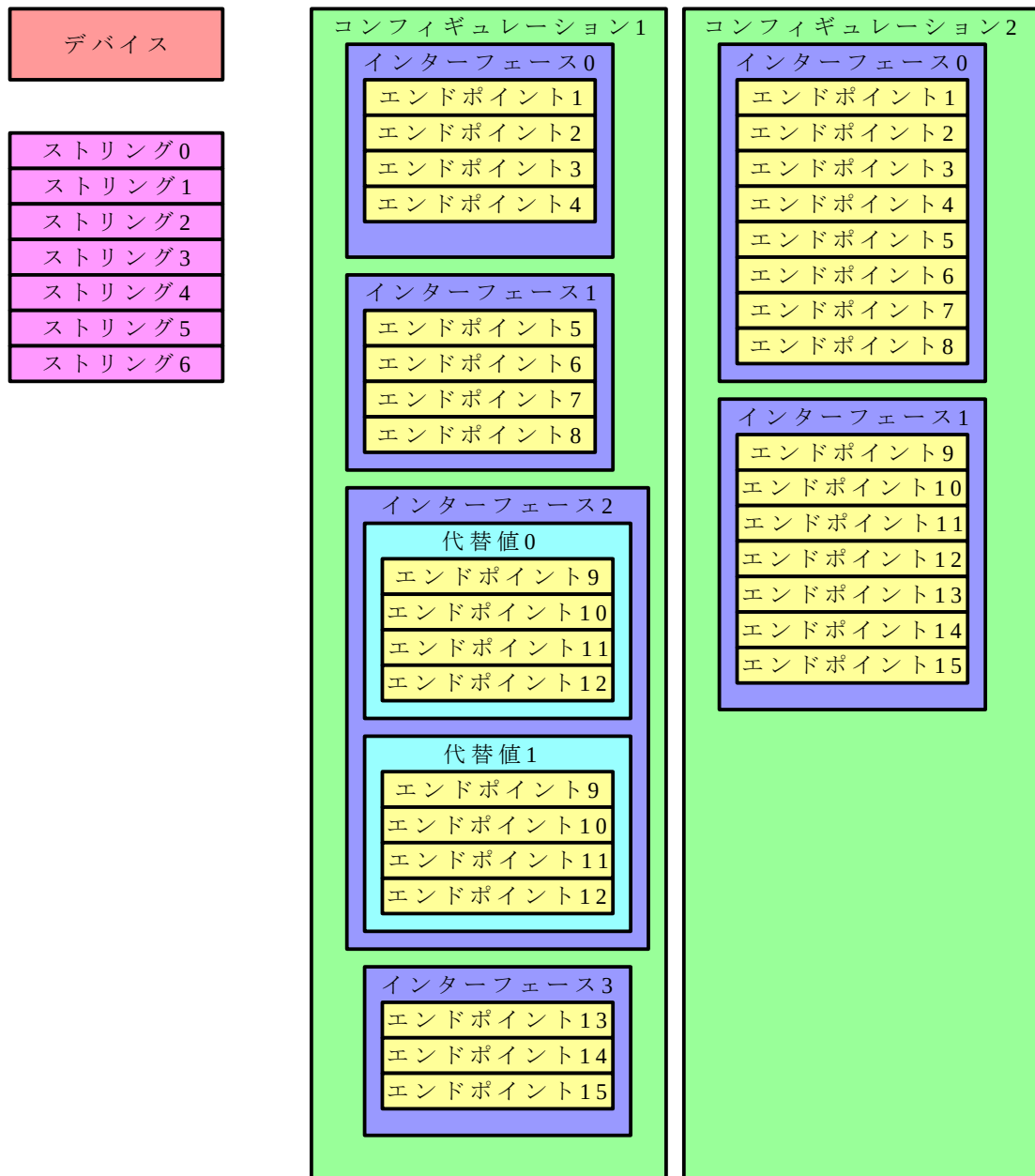
形式	値
DEVICE	1
CONFIGURATION	2
STRING	3
INTERFACE	4
ENDPOINT	5

で構成もインターフェースもひとつだけで代替値の選択もできません、Stringディスクリプタも必須ではないので外せます。尚、コンフィギュレーションの 0 番は未構成を示す番号です。

1.18 転送の種類

先に説明したコントロール転送と他にインタラプト転送、バルク転送、アイソクロナス転送の 4 種類があります。転送の種類はエンドポイントの属性なのでエンドポイントディスクリプタの定義値で固定されています。エ

図 1.10 ディスクリプタ構成 例 1



エンドポイント 0 はエンドポイントディスクリプタを持たない特別のエンドポイントでコントロール転送のみが可能です。

各転送の概要は表 1.14 のとおりです。転送周期のあるものはエンドポイントディスクリプタに定義します。コントロール転送は双方向ですが他の転送は片方向なのでエンドポイントディスクリプタに転送方向を定義します。表 1.14 に示した「最大 byte/packet」を上限として各エンドポイントの packets 容量をエンドポイントディスクリプタに定義します。Data パケットの使い方は基本的に偶数パケットを始めに偶数パケット、奇数パケット、偶数パケット・・・と送ります。例外としてアイソクロナス転送では偶数パケットだけを使います、アイソクロナス転送では Handshake パケットのやりとりがないので Data パケットの再送が発生せず奇数と偶数でパケットの順序を監視する必要がありません。またコントロール転送の Status ステージは奇数パケットに決まっ

表 1.14 転送の種類

名前	最大 byte/packet		応答 パケット	転送 周期	特徴
	full	low			
コントロール	8/16/32/64	8	あり	不定	デバイス制御、双方向
バルク	64	8	//	//	実時間性は低いが大量のデータ転送ができる。
インタラプト	64		//	1 ~ 255ms	データ量は少ないが実時間で転送できる。
アイソクロナス	1023		なし	1ms	一定の周期でデータ転送ができる。

ています。

1.19 情報源

USB コントローラの論理設計をするために下記の資料をまとめるかたちで自分の疑問点を補足する形で書いてきました、詳細を知りたい方は下記の資料を御覧下さい。

転送方式の詳細はルネサスの資料が図が豊富で直感的に分かり易いですが文字の説明は少ないです。

1.19.1 日本語

ルネサス

講習会の OHP の資料を PDF にしたものとされます。

- USB の基礎知識

http://www.renesas.com/jpn/products/mpumcu/specific/usb_mcu/pdf/jusbkiso.pdf

- USB の仕様 プロトコルレイヤの概要

http://www.renesas.com/jpn/products/mpumcu/specific/usb_mcu/pdf/jchap8.pdf

- USB の仕様 デバイスフレームワークスの概要

http://www.renesas.com/jpn/products/mpumcu/specific/usb_mcu/pdf/jchap9.pdf

1.19.2 英語

細かいところはこれを見ないと分かりません、先の日本語の資料も併用すれば英文が苦手でも結構理解できます。

仕様書 <http://www.usb.org/developers/docs/>

ここからダウンロードして解凍すると PDF が入っています。